

Witty Data Interview Assessment

NDIS Invoice Management System (Rebuild + AI Invoice Extraction)

1 Table of Contents

2	Quick Start (Recommended Reading Order)	2
3	Objective	2
4	Required Technology Stack	3
5	Required Third-Party Dependencies	3
6	Reference System	4
7	System Overview	4
8	Database ERD	4
9	Primary Requirements (Must Complete)	6
9.1	Participant (Client) Management	6
9.2	Provider Management	6
9.3	Invoice Management	7
9.4	Rate Set Management	8
9.5	NDIS Excel Import	8
9.5.1	Category	9
9.5.2	Support Item	9
9.5.3	Support Item Attributes	9
9.5.4	Pricing Regions	9
9.5.5	Price Table	10
10	Secondary Requirements (Junior Optional, Senior Must Complete)	10
10.1	System User Management	10
10.2	Authentication	10
10.3	Role Based Access Control (RBAC)	11
10.4	Audit Logging	11
11	Extended Requirements: AI-based PDF Invoice Extraction and Creation	12
11.1	Invoice Upload	13
11.2	File Storage	13

11.3	Invoice Data Extraction.....	13
11.4	Invoice Draft Creation.....	14
12	API Design.....	16
13	Example API Response Formats.....	17
13.1	Login Response	17
13.2	Participant List Response	17
13.3	Invoice Response.....	17
13.4	Invoice Response (Error).....	18
14	Suggested Next.js Project Folder Structure	18
15	Evaluation Criteria.....	19
16	Deliverables	20
17	Expected Time Commitment	20
18	What NOT to do	21

2 Quick Start (Recommended Reading Order)

1. Objective
2. Primary Requirements (Must Complete) (Section 9)
3. Invoice Management (Section 9.3) ★
4. NDIS Excel Import (Section 9.5) ★
5. Secondary Requirements (Junior Optional, Senior Must Complete) (Section 10) ★
6. Deliverables (Section 16)
7. Expected Time Commitment (Section 17)

3 Objective

The objective of this assignment is to evaluate a candidate's **ability to analyze, design, and implement** a production-like system based on incomplete information.

This document intentionally describes more functionality than most candidates can complete. We expect **prioritization, trade-off decisions, and clean implementation**—not completion of every feature.

Candidates are required to recreate a simplified **NDIS invoicing platform** by studying the following:

- Reference system
- ERD and database schema
- Excel import logic
- UI screenshots

Key Capabilities Assessed

- Reverse engineering requirements from an existing system and translating them into clear implementation logic
- Designing a maintainable and scalable system architecture
- Implementing relational database models with proper constraints and relationships
- Building a modern web application using Next.js
- Implementing business logic and validation based on inferred requirements
- Handling structured data imports (e.g., Excel parsing, validation, transformation)
- Designing AI-assisted document extraction workflows

Candidates are not expected to complete every requirement. Prioritize primary requirements. The use of **AI-powered coding assistants**, such as Claude Code or OpenAI Codex, is **highly recommended**.

4 Required Technology Stack

- Node.js v24.18.0
- Next.js v16.2.10
- PostgreSQL (PostgreSQL can be run locally or via Docker)
- TypeScript
- MinIO
- OpenRouter or OpenAI
- TailwindCSS

5 Required Third-Party Dependencies

Candidates must use the following core dependencies:

- @ant-design/icons ^6.3.2
- @ant-design/nextjs-registry ^1.3.0
- antd ^6.5.1
- argon2 ^0.44.0
- bignumber.js ^11.1.5
- clsx ^2.1.1
- dayjs ^1.11.21
- kysely ^0.29.3
- lodash ^4.18.1
- minio ^8.0.7
- openai ^6.47.0 (or @openrouter/sdk ^0.13.65)
- pg ^8.22.0

- react-icons ^5.7.0
- xlsx ^0.18.5

Candidates may install additional helper libraries, if necessary, but the above dependencies must be used for the core system implementation.

6 Reference System

Candidates must study the following reference system to understand expected behaviour:

- **URL:** <https://my-ndis.wittydata.dpdns.org>
- **Username:** test@wittydata.com
- **Password:** Test1234

7 System Overview

The application is a NDIS (National Disability Insurance Scheme) invoicing system used by plan managers to:

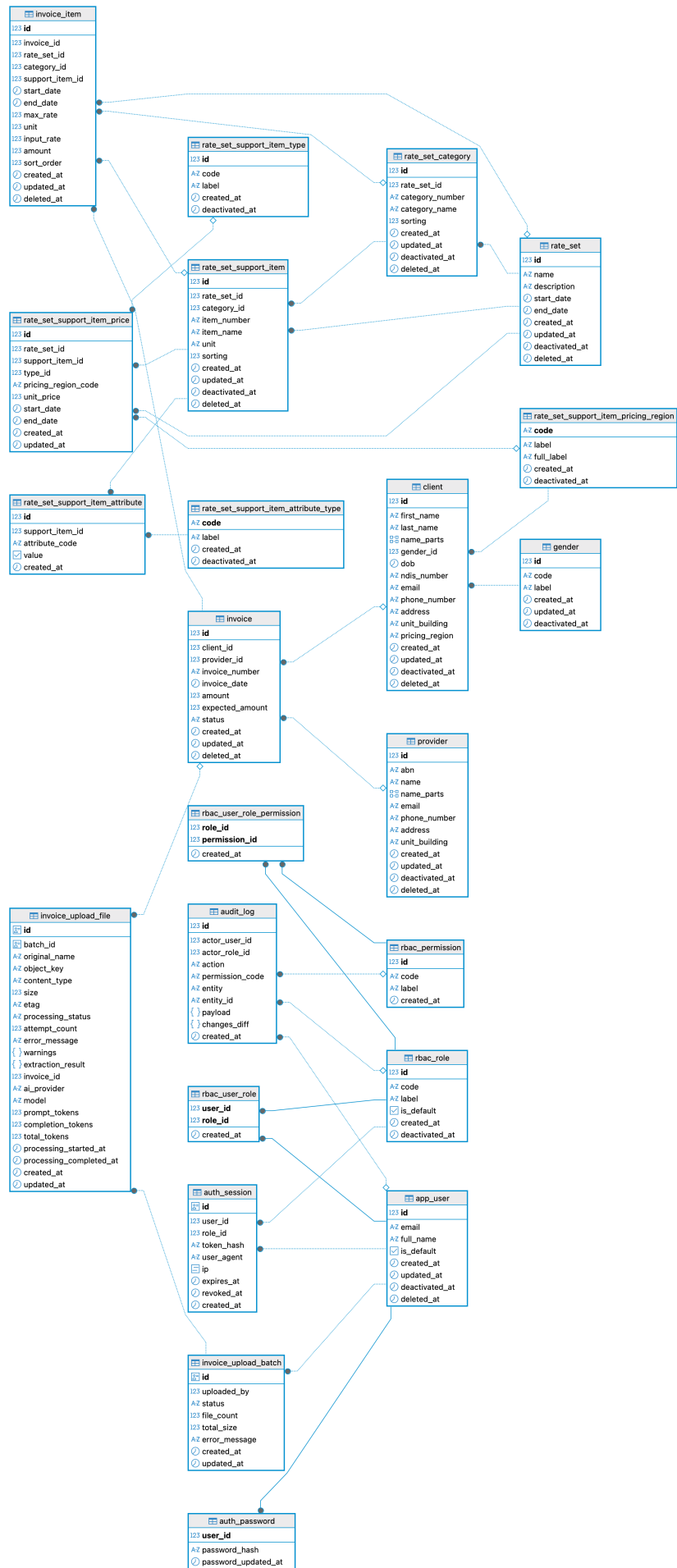
- Manage participant profiles
- Manage providers
- Generate invoices
- Import official NDIS support item rates

Core entities include:

- Clients (Participants)
- Providers
- Invoices
- Invoice Items
- NDIS Rate Sets
- Support Categories
- Support Items
- Regional Pricing

8 Database ERD

The following ERD diagram describes the primary data model used by the system (refer to the provided [db schema.sql](#) for the complete database schema and [db seed.sql](#) for sample data).



9 Primary Requirements (Must Complete)

Candidates are required to implement the following modules with an emphasis on:

- Functional UI components (e.g., forms, tables, basic interactions)
- Usability and correctness of features and data handling
- Clear structure and maintainable code

Notes:

- Pixel-perfect design is not required
- Basic styling is sufficient, but the UI should be clean and usable
- Focus on delivering a working, complete solution over visual polish

Deletion Strategy:

- All deletions must be implemented as **soft deletes** using `deleted_at`
- Deleted records must not appear in default queries
- Historical data (e.g., invoices) must remain intact

9.1 Participant (Client) Management

Users must be able to:

- Create participants
- Edit participants
- List participants
- View participant details

Validation rules:

- `first_name` – **Required, must not be empty after trimming**
- `last_name` – **Required, must not be empty after trimming**
- `gender_id` – **Required** (references `gender.id`)
- `dob` – **Required**
- `ndis_number` – **Required, digits only, max 16 digits**
- `email` – **Required, must be valid email address**
- `phone_number` – **Optional, digits only, must contain 3–16 digits**
- `address` – **Required, must not be empty after trimming**
- `unit_building` – **Optional, if provided must not be empty after trimming**
- `pricing_region` – **Required**

9.2 Provider Management

Users must be able to:

- Create providers
- Edit providers
- List providers
- View provider details

Validation rules:

- abn – **Required, digits only, max 11 digits**
- name – **Required, must not be empty after trimming**
- email – **Required, must be valid email address**
- phone_number – **Optional, digits only, must contain 3–16 digits**
- address – **Required, must not be empty after trimming**
- unit_building - **Optional, if provided must not be empty after trimming**

9.3 Invoice Management

Users must be able to:

- Create invoices and invoice items
- Edit invoices and invoice items
- List invoices
- View invoice details

Invoice validation rules:

- client_id – **Required** (references client.id)
- provider_id – **Required** (references provider.id)
- invoice_number – **Required, unique per provider_id**
- invoice_date – **Required**
- amount – **Decimal, derived from sum of invoice_item.amount**
- expected_amount – **Required, decimal, must be equal to amount**
- status – **Required**
 - **drafted** (when “Save as Draft”, allow validation errors and any valid values shall be saved)
 - **completed** (when “Saved” successfully without validation errors)

Draft mode:

- Allows incomplete or invalid data to be saved, but must still include:
 - invoice_number
 - invoice_date
 - expected_amount
- Final validation is enforced only when status = completed

Invoice item validation rules (allow validation errors when saving as draft):

- invoice_id – **Required** (references invoice.id)
- rate_set_id – **Required** (references rate_set.id)
 - Use the selected **invoice_item.start_date** and **invoice_item.end_date** date range to find a single overlapping rate set (by checking against **rate_set.start_date** and **rate_set.end_date** range)
 - A rate set is considered matching if:
 - $\text{rate_set.start_date} \leq \text{invoice_item.end_date}$ AND (rate_set.end_date IS NULL OR $\text{rate_set.end_date} \geq \text{invoice_item.start_date}$)
 - Show an error when two or more rate sets match the selected date range

- category_id – **Required** (references rate_set_category.id)
- support_item_id – **Required** (references rate_set_support_item.id)
- start_date – **Required, start of day**
- end_date – **Required, end of day**
- max_rate – **Required**
 - Use the **selected invoice_item.rate_set_id, invoice_item.support_item_id, invoice_item.start_date, invoice_item.end_date** and **client.pricing_region** to find a matching **rate_set_support_item_price.unit_price**
 - When there are multiple matches, rank by: latest **rate_set_support_item_price.start_date**, then latest finite **rate_set_support_item_price.end_date**, then highest **rate_set_support_item_price.id**
- unit – **Required, decimal**
- input_rate – **Required, decimal**
- amount – **Required, amount = unit * input_rate**

Source of truth rules:

- All monetary values must use **decimal values with 2 decimal places**
- Amounts must be **rounded to 2 decimal places (standard rounding)**
- **invoice.amount** must always be derived from invoice items (server-side) - **recalculated on every save**
- Client/pricing logic must be **enforced in the backend/API layer**, frontend logic exists only to improve the user experience and minimize input errors
- All item-level validation enforced before completion

9.4 Rate Set Management

The system stores official NDIS price limits with the following tables:

- rate_set
- rate_set_category
- rate_set_support_item
- rate_set_support_item_price

Users must be able to:

- Create rate sets
- Edit rate sets
- List rate sets
- View rate set details
- Upload NDIS Excel pricing file
- Process and import all worksheets
- Populate database tables

9.5 NDIS Excel Import

Candidates must implement a NDIS Excel pricing importer based on the provided mapping (refer to the provided [ndis_excel_import_logic.sql](#) file for the complete specification).

Key rules include:

- Excel Column A (Support Item Number) should be treated as the global unique identifier when processing the Excel file. Deduplicate with Excel Column F (Support Category Number (PACE)), Excel Column K (Start Date) and Excel Column L (End Date) if needed
- The import must be **idempotent**, meaning:
 - Re-importing the same file produces no **changes**
 - Data comparison must be based on business keys, not row order
 - Changes must be detected at field level
 - New lines should be added
 - Existing lines should be updated
 - Missing lines should be marked as deleted or deactivated by setting the deleted_at or deactivated_at timestamp accordingly whenever applicable

9.5.1 Category

- Excel Column F → category_number
- Excel Column H → category_name
- Constraints:
 - Unique category_number per rate_set
 - Categories sorted numerically

9.5.2 Support Item

- Excel Column A → item_number
- Excel Column B → item_name
- Excel Column I → unit
- Constraints:
 - unique(rate_set_id, category_number, item_number)

9.5.3 Support Item Attributes

- Excel columns map to attributes:

Excel Column	Attribute Code
J	IS_QUOTE_REQUIRED
W	IS_NF2F_SUPPORT_PROVISION
X	IS_PROVIDER_TRAVEL
Y	IS_SHORT_NOTICE_CANCEL
Z	IS_NDIA_REQUESTED_REPORTS

- Values:
 - Yes / Y (case-insensitive) → true
 - Others → false

9.5.4 Pricing Regions

- Excel columns map to attributes:

Excel Column	Region Code	Label	Full Label
M	ACT	ACT	Australian Capital Territory
N	NSW	NSW	New South Wales

O	NT	NT	Northern Territory
P	QLD	QLD	Queensland
Q	SA	SA	South Australia
R	TAS	TAS	Tasmania
S	VIC	VIC	Victoria
T	WA	WA	Western Australia
U	REMOTE	Remote	Remote
V	VERY_REMOTE	Very Remote	Very Remote

- Each region produces a price entry

9.5.5 Price Table

- Each row generates entries in **rate_set_support_item_price** table
- Constraints:
 - unique (rate_set_id, support_item_id, type_id, pricing_region_code, start_date, end_date)

10 Secondary Requirements (Junior Optional, Senior Must Complete)

Junior candidates who wish to demonstrate advanced capabilities may attempt the following modules. Senior candidates are expected to implement them:

10.1 System User Management

Users with the required permissions must be able to:

- Create system users
- Edit system users
- List system users
- View system user details
- Change **their own password** (current logged-in user only)

Validation rules:

- email – **Required, must be valid email address**
- full_name – **Required, must not be empty after trimming**
- role_id – **Required** (references rbac_role.id)

10.2 Authentication

Users must be able to:

- Log in using **email and password**
- Maintain an **authenticated session**
- Log out securely

Database tables involved:

- app_user
- auth_password
- auth_session

Requirements:

- Secure password hashing (argon2)
- Session persistence in database
- Session expiration and validation handling

Expected considerations:

- Basic input validation
- Secure handling of credentials (no plaintext storage)
- Proper session invalidation on logout

10.3 Role Based Access Control (RBAC)

The system implements RBAC using the following tables to achieve full user access control:

- rbac_role
- rbac_permission
- rbac_user_role
- rbac_user_role_permission

Example roles:

- Super Admin
- Billing Officer
- Data Entry
- Auditor

Permission scope includes:

- Participants (Read, Write, Delete)
- Providers (Read, Write, Delete)
- Invoices (Read, Write, Delete)
- Rate Sets (Read, Write, Delete, Import)
- Users (Read, Write, Delete)
- User Roles (Read, Write, Delete)
- Genders (Read, Write, Delete)
- Auth Sessions (Read, Delete, Revoke)
- Audit Logs (Read)

Requirements:

- Enforce permissions at the backend/API layer (not UI-only)
- Prevent unauthorized access to restricted actions

10.4 Audit Logging

The system must record audit logs for all implemented modules for key actions, including:

- Creating records
- Updating records

- Deleting records

Required fields to capture:

- **User ID** performing the action
- **User role(s)** at the time of action
- **Action type** (Create / Update / Delete)
- **Permission used** to authorize the action
- **Entity name** (e.g., invoice, participant)
- **Record ID** of the affected entity
- **Timestamp** (server-generated)
- **Original payload** (before change, if applicable)
- **Changes (diff)** — only modified fields (for updates)

Requirements:

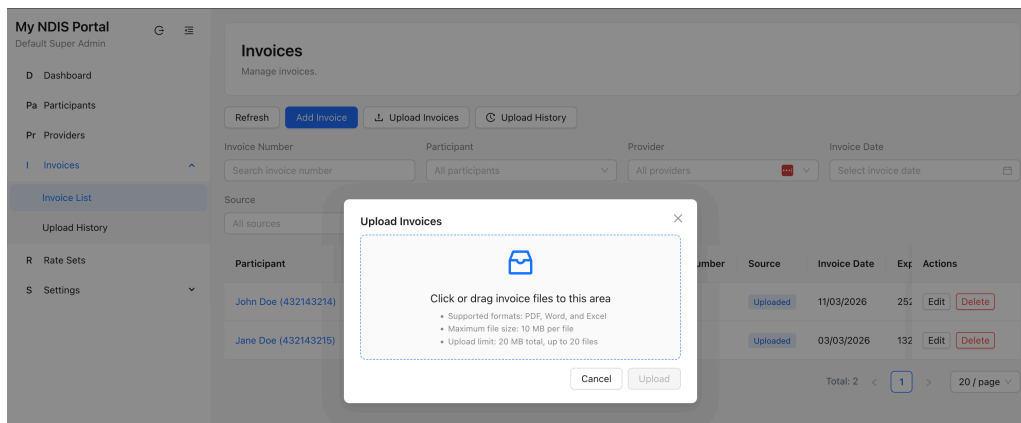
- Logs must be **created at the backend/API layer** (not client-side)
- Audit logging must be **reliable and consistent across all modules**
- Logging should **not block the main operation** (non-breaking if logging fails, but should be handled gracefully)

Expected considerations:

- Avoid logging **sensitive data** (e.g., passwords, tokens)
- Ensure logs are **queryable** (e.g., filter by user, entity, date)
- Use a **structured format** (e.g., JSON) for payload and changes
- Maintain **consistency in naming and schema design**

11 Extended Requirements: AI-based PDF Invoice Extraction and Creation

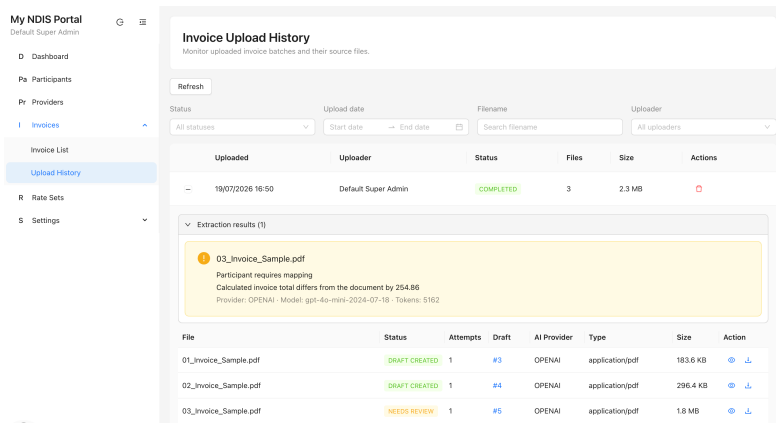
Candidates who wish to demonstrate advanced capabilities must implement a tool that extracts service details from uploaded PDF invoices using the **OpenRouter unified API** or **OpenAI API Platform** and use the extracted data to draft an invoice in the system. (Use the provided [01 Invoice Sample.pdf](#), [02 Invoice Sample.pdf](#) files for testing and validation)



11.1 Invoice Upload

Introduce an upload button in the Invoice module listing for uploading **PDF invoices**:

- Single file upload
- Multiple files in a batch (Optional)
- Invoice upload history (Optional)



11.2 File Storage

The uploaded files must be stored in MinIO.

11.3 Invoice Data Extraction

Data shall be extracted from the uploaded PDF invoice using **OpenRouter unified API** or **OpenAI API Platform** as follows:

- Use a **PDF parser** to parse the contents of the PDF or convert the PDF to base64 (Tip: Avoid sending the entire PDF as binary to reduce token usage.)
- Design and create a prompt that instructs the model to extract key information from the PDF contents into structured JSON as follows:

```
{
  "participant_name": "Jane Doe",
  "participant_ndis_number": "432143215",
  "provider_name": "SERENITY LIFE BALANCE ADVISORY PTY LTD",
  "provider_abn": "73 628 557 755",
  "invoice_number": "INV0705",
  "invoice_date": "2026-03-03",
  "stated_invoice_total": "1329.18",
  "line_items": [
    {
      "unit": "6",
      "invoiced_rate": "95.07",
      "stated_amount": "570.42",
      "service_end_date": "2026-02-28",
      "service_start_date": "2026-02-07",
      "support_item_number": "04_105_0125_6_1"
    },
    {
      "unit": "6",
      "invoiced_rate": "67.56",
```

```

        "stated_amount": "405.36",
        "service_end_date": "2026-02-19",
        "service_start_date": "2026-02-12",
        "support_item_number": "04_104_0125_6_1"
    },
    {
        "unit": "5",
        "invoiced_rate": "67.56",
        "stated_amount": "337.80",
        "service_end_date": "2026-02-20",
        "service_start_date": "2026-02-20",
        "support_item_number": "04_104_0125_6_1"
    },
    {
        "unit": "12",
        "invoiced_rate": "1.30",
        "stated_amount": "15.60",
        "service_end_date": "2026-02-01",
        "service_start_date": "2026-02-01",
        "support_item_number": "04_590_0125_6_1"
    }
]
}

```

11.4 Invoice Draft Creation

The extracted data shall be transformed and mapped accordingly to facilitate the creation of an invoice draft

Participant Mapping:

1. Use the extracted **participant_ndis_number** to match the **client.ndis_number**
2. When there is only a single match, use the **client.id** as **invoice.client_id**
3. Skip to Provider Mapping when there is no match
4. When there are multiple matches, tokenize **participant_name** and match it with **client.name_parts**
5. Rank the results from highest number of name parts matches to the lowest, use **client.id** as the tiebreaker
6. Use the **client.id** of the highest ranked result as **invoice.client_id**

Provider Mapping:

1. Use the extracted **provider_abn** to match the **provider.abn**
2. When there is only a single match, use the **provider.id** as **invoice.provider_id**
3. Skip to Invoice Mapping when there is no match
4. When there are multiple matches, tokenize **provider_name** and match it with **provider.name_parts**
5. Rank the results from highest number of name parts matches to the lowest, use **provider.id** as the tiebreaker
6. Use the **provider.id** of the highest ranked result as **invoice.provider_id**

Invoice Mapping:

1. Use the extracted **invoice_number** as **invoice.invoice_number**
2. Use the extracted **invoice_date** as **invoice.invoice_date**
3. Use the extracted **stated_invoice_total** as **invoice.expected_amount**

Rate Set Mapping:

1. Refer to **Section 9.3 (Invoice Management)** for more information
2. Use the extracted **service_start_date** and **service_end_date** date range to determine whether it overlaps with or is fully contained within the **rate_set.start_date** and **rate_set.end_date** date range
3. When there is only a single match, use the **rate_set.id** as **invoice_item.rate_set_id**
4. Skip to Invoice Item Mapping when there is no match or multiple matches

Support Item and Category Mapping:

1. Use the extracted **support_item_number** to match the **rate_set_support_item.item_number** (filter the **rate_set_support_item.rate_set_id** with the previously matched **rate_set.id**)
2. When there is only a single match, use the **rate_set_support_item.id** as **invoice_item.support_item_id** and **rate_set_support_item.category_id** as **invoice_item.category_id**
3. Skip to Invoice Item Mapping when there is no match or multiple matches

Invoice Item Mapping:

1. Use the extracted **service_start_date** as **invoice_item.start_date**
2. Use the extracted **service_end_date** as **invoice_item.end_date**
3. Use the previously matched **invoice_item.rate_set_id**, **invoice_item.support_item_id**, **invoice_item.start_date**, **invoice_item.end_date** and **client.pricing_region** to find a matching **rate_set_support_item.price.unit_price** to be used as **invoice_item.max_rate** (Refer to **Section 9.3 (Invoice Management)** for more information)
4. Use the extracted **unit** as **invoice_item.unit**
5. Use the extracted **invoiced_rate** as **invoice_item.input_rate**
6. Use the extracted **unit** multiplied by **invoiced_rate** as **invoice_item.amount**

Once the extracted data is transformed and mapped correctly, proceed to create an invoice draft based on the following guidelines:

Invoice Table:

Column	Description
client_id	Nullable (When no participant matched)
provider_id	Nullable (When no provider matched)
invoice_number	Required. Cannot create invoice draft when null
invoice_date	Nullable (When it cannot be extracted)
expected_amount	Nullable (When it cannot be extracted)
amount	Nullable (When it cannot be calculated from invoice_item.amount)
status	drafted

Invoice Item Table:

Column	Description
rate_set_id	Nullable (When no rate set or multiple rate sets matched)

category_id	Nullable (When no rate set or multiple rate sets matched, or when no support item or multiple support items matched)
support_item_id	Nullable (When no rate set or multiple rate sets matched, or when no support item or multiple support items matched)
start_date	Nullable (When it cannot be extracted)
end_date	Nullable (When it cannot be extracted)
max_rate	Nullable (When it cannot be matched)
unit	Nullable (When it cannot be extracted)
input_rate	Nullable (When it cannot be extracted)
amount	Nullable (When it cannot be calculated from unit * input_rate)

AI extraction must:

- Handle partial or missing fields
- Avoid crashing when extraction is incomplete
- Allow user correction before saving
- Not assume perfect input format

Accuracy Expectation

- Perfect extraction accuracy is not required
- Evaluation is based on:
 - Reasonable field mapping
 - Handling of incomplete/ambiguous data
 - Ability to recover via user correction

12 API Design

Candidates should implement a clear **RESTful** API design with an emphasis on:

- Proper validation
- Error handling
- Clean structure (service/repository separation preferred)

API Design Requirements

- All business logic must be implemented in the backend/API layer
- The backend/API layer is the sole source of truth
- API should be structured and consistent (REST or RPC)

Error Handling

- Validation errors must return clear, structured responses
- System errors should not expose internal details
- API responses should be consistent across modules

All API responses should follow a consistent structure:

- Success: { data, meta? }
- Error: { error: { code, message, details? } }

Example API endpoints include:

- POST /api/login
- GET /api/clients
- POST /api/clients
- GET /api/providers
- POST /api/invoices
- POST /api/rate-sets/import

13 Example API Response Formats

13.1 Login Response

```
{
  "data": {
    "user": {
      "id": 1,
      "email": "test@wittydata.com",
      "full_name": "Test User"
    },
    "session_token": "abc123token",
    "expires_at": "2026-01-01T00:00:00Z"
  }
}
```

13.2 Participant List Response

```
{
  "data": [
    {
      "id": 1,
      "first_name": "John",
      "last_name": "Doe",
      "ndis_number": "4300123456",
      "pricing_region": "VIC"
    }
  ],
  "total": 1
}
```

13.3 Invoice Response

```
{
  "data": {
    "id": 101,
    "client_id": 1,
```

```

    "provider_id": 1,
    "invoice_number": "INV-2026-001",
    "invoice_date": "2026-02-01",
    "amount": 450.00,
    "items": [
      {
        "support_item_id": 22,
        "description": "Daily Living Support",
        "unit": 3,
        "unit_price": 150.00,
        "amount": 450.00
      }
    ]
  }
}

```

13.4 Invoice Response (Error)

```

{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "One or more fields are invalid.",
    "details": {
      "invoice_number": ["Invoice number is required"]
    }
  }
}

```

14 Suggested Next.js Project Folder Structure

Candidates should organise the project with a clean and maintainable structure such as:

```

/app
/login
/dashboard
/clients
/providers
/invoices
/rate-sets

/modules
/client
/provider
/invoice
/rate-set

/services
auth.service.ts

```

invoice.service.ts
ai-extraction.service.ts

/repositories
client.repository.ts
invoice.repository.ts

/db
schema.ts
migrations/

/lib
s3.ts
ai.ts
auth.ts

15 Evaluation Criteria

All candidates are evaluated relative to other candidates of similar experience level. Therefore, candidates are encouraged to focus on completing the primary requirements and only attempt the extended requirements after that.

Primary Requirements — **100%**

- Architecture design – 20%
- Database implementation – 15%
- Backend/API design – 20%
- Frontend usability – 15%
- NDIS Excel import – 20%
- Code quality and maintainability – 10%

Secondary Requirements — **100%**

- Architecture design – 20%
- Database implementation – 20%
- Backend/API design – 25%
- Frontend usability – 25%
- Code quality and maintainability – 10%

AI Invoice Extraction & Creation — **100%**

- MinIO upload implementation – 10%
- Frontend usability – 10%
- AI integration design – 15%
- AI prompt design – 15%
- Extraction accuracy – 5%
- Data mapping and validation – 25%
- Invoice draft creation – 20%

For senior candidates, we will also evaluate:

- How you structure the codebase
- Trade-offs you make (document in README)
- Handling of edge cases
- Data consistency strategies
- Performance considerations

What a Strong Submission Looks Like:

- Clear separation of concerns (API, services, DB)
- Correct handling of edge cases and validation
- Consistent API structure and error handling
- Thoughtful trade-offs documented in README
- Clean, readable, and maintainable code

16 Deliverables

- Git repository
- Database migration scripts
- README that includes:
 - Setup instructions
 - Assumptions made
 - Trade-offs
 - What is incomplete
 - Known limitations or future improvements

17 Expected Time Commitment

Submissions will not be penalized for incomplete optional features if primary functionality is implemented well.

- Primary requirements: **16–24 hours**
- Secondary requirements: **12–20 hours**
- AI Invoice Extraction and Creation: **12–20 hours**

Candidates are **not expected to complete every requirement**. Prioritize:

- Primary backend correctness
- Invoice + rate logic
- User Management, Authentication, RBAC, Audit Logging
- Then extended AI invoice extraction and creation

Suggested approach:

- Spend ~60% on backend
- Spend ~40% on frontend

18 What NOT to do

- Over-engineer infrastructure (no need for microservices)
- Focus excessively on UI polish
- Implement unnecessary features not listed

We're excited to see your approach and thought process. Good luck!